

Detecção de Deadlocks e Violação da Ordem de Aquisição de Travas

Baseado em DEFINITION.md

Universidade Federal do Rio Grande do Norte

16 de julho de 2025

Integrantes

Nome	E-mail
Corpo docente	
Samuel Xavier de Souza	samuel.xavier@ufrn.br
Wedson Torres de Almeida Filho	wedson.almeida@dca.ufrn.br
Corpo discente	
Vinicius de Lima Duarte Sampaio	vinicius.lima.115@ufrn.edu.br
Emanuel Rawa Gurgel de Araujo Freitas	emanuelrawa@gmail.com
Arthur Jose Arruda Skeete	arthurjoskeete@gmail.com
Yuri Maximiliano Brasileiro Santos	yurimax322@gmail.com
José Henrique Targino Dias Góis	ze.gois.00@gmail.com

Sumário

- 1 Introdução e Objetivo
- 2 Fundamentos Teóricos
- 3 Implementação por Arquivo
- 4 Exemplo de Execução
- 5 Conclusão e Extensões

Objetivo Geral

- Estender mutexes didáticos para detectar deadlocks e violações de ordem.
- Inspirado no lockdep do Linux.
- Abordagens: Grafo de espera (reativo) ou verificação de ordem (preventivo).

Requisitos Principais

- Base em mutex C.
- Estruturas: Dono atual, fila de espera, ordem por thread.
- Implementar pelo menos uma abordagem (extra para ambas).
- Foco aqui: Lockdep-style (preventivo); grafo de espera como extensão.

Conceitos Chave

- Deadlock: Espera circular (condições de Coffman).
- Grafo de Espera: Nodes=threads/resources; ciclo=deadlock real (DFS).
- Lockdep-Style: Grafo dependências (nodes=locks, edges=ordem); ciclo=potencial inversão.

pthread_interpose.c: Hooking Pthread

- Recebe endereço dos símbolos dinâmicos a serem ligados em tempo de execução;
- Envelope das funções reais por funções de interposição.

pthread_interpose.c: Hooking Pthread

- Recebe endereço dos símbolos dinâmicos a serem ligados em tempo de execução;

```
9 static int (*real_pthread_mutex_lock)(pthread_mutex_t*) = NULL;
10 static int (*real_pthread_mutex_unlock)(pthread_mutex_t*) = NULL;
11 static int (*real_pthread_mutex_trylock)(pthread_mutex_t*) = NULL;
12
13 /// This interposes the real pthread mutex functions to add lockdep validation
14 static void init_real_functions(void)
15 {
16     if (!real_pthread_mutex_lock) {
17         real_pthread_mutex_lock = dlsym(RTLD_NEXT, "pthread_mutex_lock");
18     }
19     if (!real_pthread_mutex_unlock) {
20         real_pthread_mutex_unlock = dlsym(RTLD_NEXT, "pthread_mutex_unlock");
21     }
22     if (!real_pthread_mutex_trylock) {
23         real_pthread_mutex_trylock = dlsym(RTLD_NEXT, "pthread_mutex_trylock");
24     }
25 }
```


pthread_interpose.c: Hooking Pthread

- Envelope das funções reais por funções de interposição.

```
37 int pthread_mutex_lock(pthread_mutex_t* mutex)
38 {
39     init_real_functions();
40     if (lockdep_enabled && !in_interpose) {
41         in_interpose = true;
42         if (!lockdep_acquire_lock(mutex)) {
43             in_interpose = false;
44         }
45         in_interpose = false;
46     }
47     int result = real_pthread_mutex_lock(mutex);
48     return result;
49 }
```

pthread_interpose.c: Hooking Pthread

- Envelope das funções reais por funções de interposição.

```
51 int pthread_mutex_trylock(pthread_mutex_t* mutex)
52 {
53     init_real_functions();
54     int result = real_pthread_mutex_trylock(mutex);
55     if (result == 0 && lockdep_enabled && !in_interpose) {
56         in_interpose = true;
57         if (!lockdep_acquire_lock(mutex)) {
58             real_pthread_mutex_unlock(mutex);
59             in_interpose = false;
60             return EBUSY;
61         }
62         in_interpose = false;
63     }
64     return result;
65 }
```

pthread_interpose.c: Hooking Pthread

- Envelope das funções reais por funções de interposição.

```
67 int pthread_mutex_unlock(pthread_mutex_t* mutex)
68 {
69     init_real_functions();
70     int result = real_pthread_mutex_unlock(mutex);
71     if (lockdep_enabled && !in_interpose) {
72         in_interpose = true;
73         lockdep_release_lock(mutex);
74         in_interpose = false;
75     }
76     return result;
77 }
```

lockdep.h: Estruturas de Dados

- `lock_node_t`: Grafo (children) + lista (next); dono (`current_held_lock`).
- `adjacency_locks_t`: Edges para dependências.
- `held_lock_t`: Pilha de holds com trace e timestamp.
- `thread_context_t`: Contexto thread com pilha held.

lockdep.h: Estruturas de Dados

- lock_node_t: Grafo (children) + lista (next); dono (current_held_lock).

```
29
30 typedef struct lock_node {
31     const void* lock_addr;
32     char lock_name[LOCKDEP_MAX_LOCK_NAME];
33     bool was_visited;
34     adjacency_locks_t* children;
35     struct lock_node* next;
36     held_lock_t* current_held_lock;
37 } lock_node_t;
```

- adjacency_locks_t: Edges para dependências.

```
40 typedef struct adjacent_locks {
41     lock_node_t* lock;
42     struct adjacent_locks* next;
43 } adjacency_locks_t;
```

lockdep.h: Estruturas de Dados

- held_lock_t: Pilha de holds com trace e timestamp.

```
45
46 typedef struct held_lock {
47     lock_node_t* lock;
48     struct timespec acquired_at;
49     void* acquisition_stack[LOCKDEP_MAX_STACK_DEPTH];
50     int stack_depth;
51     thread_context_t* thread_context;
52     struct held_lock* next;
53 } held_lock_t;
```

- thread_context_t: Contexto thread com pilha held.

```
55
56 typedef struct thread_context {
57     pthread_t pthread_id;
58     pid_t thread_id;
59     held_lock_t* held_locks;
60     char thread_name[32];
61     struct thread_context* next;
62 } thread_context_t;
```

lockdep_core.c: Lógica Central

- lockdep_init: Inicializa
- lockdep_acquire_lock: Registra, adiciona dep, checa ciclo (DFS).
- lockdep_release_lock: Libera, remove

```

300         "=====\\n"
301         "[LOCKDEP] *** DEADLOCK DETECTED ***\\n"
302         "=====\\n");
303
304     fprintf(stderr, "\\n[LOCKDEP] Deadlock cycle detected:\\n");
305     for (size_t i = 0; i < cycle_path_size; i++) {
306         lock_node_t* current_lock = cycle_path[i];
307         fprintf(stderr, "    [%zu] %s\\n", i + 1, current_lock->lock_name);
308
309         if (current_lock->current_held_lock) {
310             held_lock_t* held = current_lock->current_held_lock;

```

lockdep_core.c: Lógica Central

- lockdep_init: Inicializa
- lockdep_acquire_lock: Registra, adiciona dep, checa ciclo (DFS).
- lockdep_release_lock: Libera, remove

```

300         "=====\\n"
301         "[LOCKDEP] *** DEADLOCK DETECTED ***\\n"
302         "=====\\n");
303
304     fprintf(stderr, "\\n[LOCKDEP] Deadlock cycle detected:\\n");
305     for (size_t i = 0; i < cycle_path_size; i++) {
306         lock_node_t* current_lock = cycle_path[i];
307         fprintf(stderr, "    [%zu] %s\\n", i + 1, current_lock->lock_name);
308
309         if (current_lock->current_held_lock) {
310             held_lock_t* held = current_lock->current_held_lock;

```


lockdep_core.c: Lógica Central

- lockdep__init: Inicializa

```
372 void lockdep_init(void)
373 {
374     const char* env = getenv("LOCKDEP_DISABLE");
375     if (env && strcmp(env, "1") == 0) {
376         lockdep_enabled = false;
377         return;
378     }
379 }
```

lockdep_core.c: Lógica Central

- lockdep_acquire_lock: Registra, adiciona dep, checa ciclo (DFS).

```
381 bool lockdep_acquire_lock(const void* lock_addr)
382 {
383     fprintf(stderr, "[LOCKDEP] Acquiring lock %p\n", lock_addr);
384     pthread_mutex_lock(&lockdep_mutex);
385
386     thread_context_t* ctx = find_thread_context(pthread_self());
387
388     lock_node_t* lock = find_or_register_lock(lock_addr);
389     add_dependency_to_graph(add_lock_to_thread_context(ctx, lock), lock);
390     const bool deadlock = has_deadlock(lock);
391     deadlock ? 42 : unvisit_locks();
392
393     pthread_mutex_unlock(&lockdep_mutex);
394     return !deadlock;
395 }
```

lockdep_core.c: Lógica Central

- lockdep_release_lock: Libera, remove

```
397 void lockdep_release_lock(const void* lock_addr)
398 {
399     fprintf(stderr, "[LOCKDEP] Releasing lock %p\n", lock_addr);
400     pthread_mutex_lock(&lockdep_mutex);
401
402     release_lock_from_thread_context(find_thread_context(pthread_self()), lock_addr);
403     pthread_mutex_unlock(&lockdep_mutex);
404 }
```

lockdep_core.c: Lógica Central

- lockdep_acquire_lock: Registra, adiciona dep, checa ciclo (DFS).

```
381 bool lockdep_acquire_lock(const void* lock_addr)
382 {
383     fprintf(stderr, "[LOCKDEP] Acquiring lock %p\n", lock_addr);
384     pthread_mutex_lock(&lockdep_mutex);
385
386     thread_context_t* ctx = find_thread_context(pthread_self());
387
388     lock_node_t* lock = find_or_register_lock(lock_addr);
389     add_dependency_to_graph(add_lock_to_thread_context(ctx, lock), lock);
390     const bool deadlock = has_deadlock(lock);
391     deadlock ? 42 : unvisit_locks();
392
393     pthread_mutex_unlock(&lockdep_mutex);
394     return !deadlock;
395 }
```

lockdep_core.c: Lógica Central

- lockdep_acquire_lock: find thread context

```
145 static thread_context_t* find_thread_context(const pthread_t pthread_id)
146 {
147     thread_context_t* ctx = thread_registry;
148     while (ctx) {
149         if (ctx->pthread_id == pthread_id) return ctx;
150         ctx = ctx->next;
151     }
152     return NULL;
153 }
```

lockdep_core.c: Lógica Central

- lockdep_acquire_lock: find or register lock

```

125 static lock_node_t* find_or_register_lock(const void* lock_addr)
126 {
127     lock_node_t* lock = lock_registry;
128
129     while (lock) {
130         if (lock->lock_addr == lock_addr) return lock;
131         lock = lock->next;
132     }
133
134     lock = smalloc(sizeof(lock_node_t));
135     lock->children = NULL;
136     lock->lock_addr = lock_addr;
137     strncpy(lock->lock_name, generate_lock_name(lock_addr), LOCKDEP_MAX_LOCK_NAME - 1);
138     lock->lock_name[LOCKDEP_MAX_LOCK_NAME - 1] = '\0';
139     lock->next = lock_registry;
140     lock->was_visited = false;
141     lock->current_held_lock = NULL;
142     return lock_registry = lock;
143 }

```

lockdep_core.c: Lógica Central

- lockdep_acquire_lock: add lock to thread context

```
155 static thread_context_t* add_lock_to_thread_context(thread_context_t* ctx,  
    lock_node_t* lock)  
156 {  
157     if (ctx) {  
  
169     } else {  
  
188     }  
189     return ctx;  
190 }  
191 }
```

lockdep_core.c: Lógica Central

- lockdep_acquire_lock: add lock to thread context

```
158     held_lock_t* new_held = smalloc(sizeof(held_lock_t));
159     new_held->lock = lock;
160     new_held->thread_context = ctx;
161     clock_gettime(CLOCK_MONOTONIC, &new_held->acquired_at);
162     capture_stack_trace(new_held->acquisition_stack, &new_held->stack_depth,
LOCKDEP_MAX_STACK_DEPTH);
163     new_held->next = ctx->held_locks;
164     ctx->held_locks = new_held;
165
166     // Update lock ownership reference
167     lock->current_held_lock = new_held;
```


lockdep_core.c: Lógica Central

- lockdep_acquire_lock: add lock to thread context

```
155 static thread_context_t* add_lock_to_thread_context(thread_context_t* ctx,  
    lock_node_t* lock)  
156 {  
157     if (ctx) {  
  
169     } else {  
  
188     }  
189     return ctx;  
190 }  
191 }
```

lockdep_core.c: Lógica Central

- lockdep_acquire_lock: add lock to thread context

```
170 // Create new thread context.
171 ctx = smalloc(sizeof(thread_context_t));
172 ctx->held_locks = smalloc(sizeof(held_lock_t));
173 ctx->pthread_id = pthread_self();
174 ctx->thread_id = get_thread_id();
175 snprintf(ctx->thread_name, sizeof(ctx->thread_name), "thread_%d", ctx->
thread_id);
176
177 ctx->held_locks->lock = lock;
178 ctx->held_locks->thread_context = ctx;
179 clock_gettime(CLOCK_MONOTONIC, &ctx->held_locks->acquired_at);
180 capture_stack_trace(ctx->held_locks->acquisition_stack, &ctx->held_locks->
stack_depth, LOCKDEP_MAX_STACK_DEPTH);
181 ctx->held_locks->next = NULL;
182
183 ctx->next = thread_registry;
184 thread_registry = ctx;
185
186 // Update lock ownership reference
187 lock->current_held_lock = ctx->held_locks;
```

lockdep_core.c: Lógica Central

- lockdep_acquire_lock: add dependency to graph

```
212 static void add_dependency_to_graph(const thread_context_t* ctx, lock_node_t* lock)
213 {
214     held_lock_t* held = ctx->held_locks->next;
215     if (held) {
216         adjacency_locks_t* adj = held->lock->children;
217         bool already_depends = false;
218         while (adj) {
219             if (adj->lock->lock_addr == lock->lock_addr) {
220                 already_depends = true;
221                 break;
222             }
223             adj = adj->next;
224         }
225         if (!already_depends) {
226             adjacency_locks_t* new_adj = malloc(sizeof(adjacency_locks_t));
227             new_adj->lock = lock;
228             new_adj->next = held->lock->children;
229             held->lock->children = new_adj;
230         }
231     }
232 }
```

t05_dining_philosophers.c: Exemplo

- Clássico problema: 5 filósofos, 5 garfos (mutexes).
- SHALL_LOCK=1: Todos pegam left first → ciclo.
- Com lockdep: Detecta preventivamente.

```
60     if (id == NUM_PHILOSOPHERS - 1) {
61         // Last philosopher picks up right fork first
62         first_fork = right_fork;
63         second_fork = left_fork;
64     } else {
65         // Other philosophers pick up left fork first
66         first_fork = left_fork;
67         second_fork = right_fork;
68     }
69 }
```

Resultado no Exemplo

- Com LD_PRELOAD: Detecta ciclo (e.g., lock_0x8050 → ... → lock_0x8050).
- Reporta: Path, tempos, traces, states, sugestão (order by addr).
- Atende preventivo; extensão para reativo via waiters.

Conclusão

- Atende lockdep-style; demonstra grafos em concurrency.
- Limitações: $O(n)$ lookups (add hash); sem fila wait full.
- Impacto: Educacional; usa IA para análise.

Extensões Futuras

- Implementar grafo espera: Add fila waiters, DFS thread-lock.
- Suporte a outros sync (condvars, RW locks).
- Otimizações: Hash para lookups; integração gdb/valgrind.

Perguntas?

Obrigado! Dúvidas?